Chapter-1 : Introduction

Database: set of ~~enter~~ inter related data.

Database management: ~~where Data managed~~ is a system for managing the Database (Insert, Delete, Update)

Relational DataBase management system:

Disadvantage of file processing system /purpose of Database Management System

1. **Data Redundancy:** একই তথ্যের multiple copy কে redundance Data বলে।
2. **Data inconsistancy:** একই person এর Data তে অসামঞ্জস্যতা থাকে না।
3. **Data isolation:** সব ফাইল থেকে আমার তথ্য খুঁজে বের করে।
4. **Difficulty in accessing data:** যেহেতু আমরা Database management system ব্যবহার করি তাই access করতে তেমন problem হয় না।
5. **Concurrent access annomalies:** at a time অনেক মানুষ একটা তথ্য ব্যবহার করতে পারে।

6. Security : User এ যাতে প্রয়োজনে ডাটা ও অন্যান্য দেখতে না পায়.

1st lab(1).

1. Creat a DB named "BUBT_DB1"

⇒ Creat Database Sohan_246

2. Creat three tables in this DB named "student", "Faculty", "course".

⇒ Creat table student (col_name1 datatype (size)---);

Name এ ব্যবহৃত
Variable Character

1. student :

Creat table student (Name varchar(35), St_Id int(2), NID int(3), Intake int(2), Section int(1), CGPA float(3.2))

Value/Data insertion in Table:

insert into table_name values (value1, value2.....valuen)

⇒ insert into Student values ("Ashraful", 1, 102, 44, 1, 3.30);

insert into Student values ("Tonny", 2, 101, 44, 1, 3.50);

" " " " ("Zakir", 33, 103, 35, 2, 3.98);

" " " " ("Uromi", 6, 104, 35, 1, 3.96);

" " " " ("Tonny", 23, 105, 44, 1, 3.50);

---

21. Faculty :

creat table Faculty (Name varchar(40), ShortCode varchar(5), course_Code ~~varch~~ varchar(7), conduct_Semester varchar(18));

⇒ insert into Faculty values ("Farha Akhter Munmun", "FAM", "CSE 319", "Summer 2019")

" " " " ("Sweety Lima", "SWL", "CSE20

5. course:

creat table course(course_Title varchar(18),

course_code (6) varchar (7))

3. a

Update Faculty Set Short_Code = "NAF"

where Name = "Nadia Afrin Ritu";

b.

Update Course set ~~Short~~ Course_code = "CSE 317"

~~where Name course code = Title =~~

"System Analysis Design";

where Course_Title = "System Analysis Design";

c. Update ~~Conduct Semester~~ Faculty set course_semester = "Fall 2020"

where course_code = "CSE 351" ~~AND~~

Name = "Nadia Afrin Ritu";

d.

ADD column :-

Alter table ~~table_Name~~ ADD column ~~col_Name~~ datatyp(size)

Alter table Student ADD column Age int(2);

i. After table Course ADD column offered_semester varchar(10) AFTER Course_Title;

LAB-2

* Creat table Student (Name varchar(35), St_ID int(2), NID int(3), Intake int(2), section int(3), CGPA float(3.2)

Value insertion

⇒ insert into student values ("Ashraful", 1, 102, 44, 1, 3.30);

.
.
.
.
.

* creat table Faculty (Name varchar(40), Short_Code varchar(5), course_code varchar(9), conduct_Semester varchar(20)

Value insertion

⇒ insert into Faculty ("Farha Akhter Munmun", "FAM", "CSE 319", "Summer 2019");

.
.
.
.
.

* Creat table course (Course_Title varchar(20),
Course_Code varchar(10));

Value insertion:

⇒ course
insert into ("Database Systems", CSE 310);

Theory

Database Applications Examples

⇒ Enterprise Information

⇒ Manufacturing

⇒ Banking and Finance

⇒ Universities

Database (DB) :-

Database is a collection/set of interrelated data.

Database Management System (DBMS):

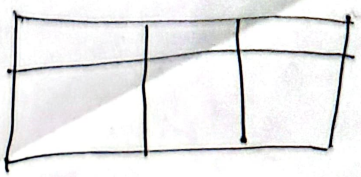
Database management system is a system for ~~management~~ ~~the database~~ managing the database.

↓

(insert, delete, Update)

Relational Database Management System (RDBMS):

Table /Relation/ Entity set



5. Purpose of Database Management System/
Disadvantage of file Processing System

1. Data redundancy

multiple copy of same data

↳ redundant Data

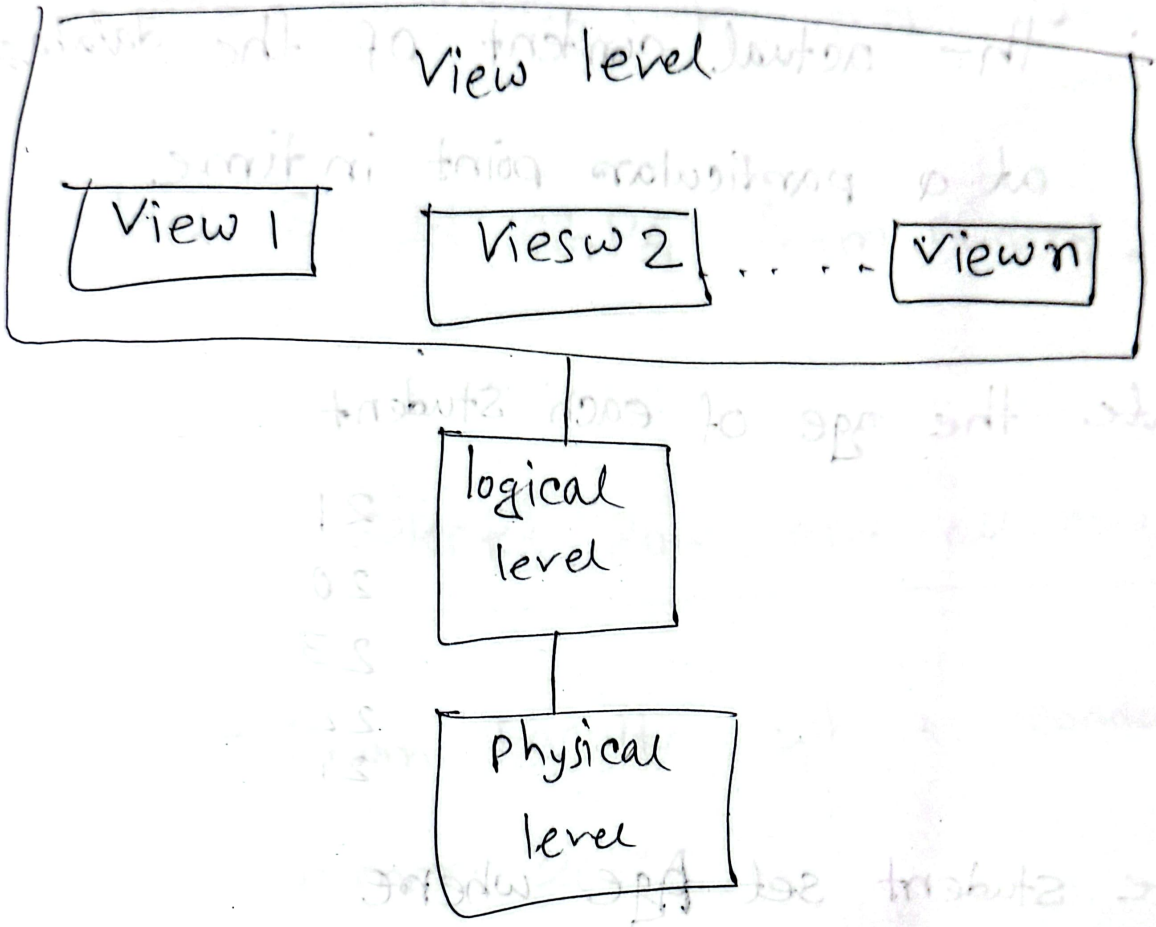
2. Data inconsisteney

1. N-A-CN-Email } file manually change
2. ... A- ... -

3. Data

View of Data

An architecture for a database system



Data Abstraction: Data abstraction is the process of hiding complex structures from users so that they can access the data easily conveniently

3 level for data abstraction :-

1. View level ⇒ Front end developers
2. logical " ⇒ Back-end, Database engineer (Table creation
3. Physical " ⇒ Admin ; and in which location How the data

Schema

The overall logical structure of the database

Instance: The actual content of the database at a particular point in time

e. Update the age of each student

21
20
23
22
21

⇒ Update student set Age where

St_Id = 1;

Update Student seq Age where St_Id = 21;

f. Find the names of all courses in the course relation

⇒ Selected_course_Title from course;

g. Select St_Id CGPA from Student;

Selected Row and all colums:

Select * From Faculty where conduct_Semesten = "Summer 2019"

Selected column & selected row:

Select Name From Faculty where Conduct_Semestera = "Summer 2019";

h. Find the names of all students whose CGPA is greater than 3.50

⇒ Select Name from Student where CGPA > 3.50;

J. Find the name and CGPA of each student order by their CGPA (Aseending Order)

⇒ Select Name, CGPA From student Order by CGPA ASC / DESC ;

k ⇒ Select Name From student where Age > 22 AND CGPA > 3.50;

l. Select ID From Student where intake = 44 OR CGPA < 3.40;

m. Select ID from Student where CGPA = 3.30 OR CGPA = 3.50 OR CGPA = 3.96;

⇒ where CGPA IN (3.30, 3.50, 3.96);

1. Data Definition Language (DDL)
2. Data manipulation " (DML)

NID Delete from student

Table: Alter table student Drop column NID

n. Delete those students from student table whose CGPA is less than 3.30

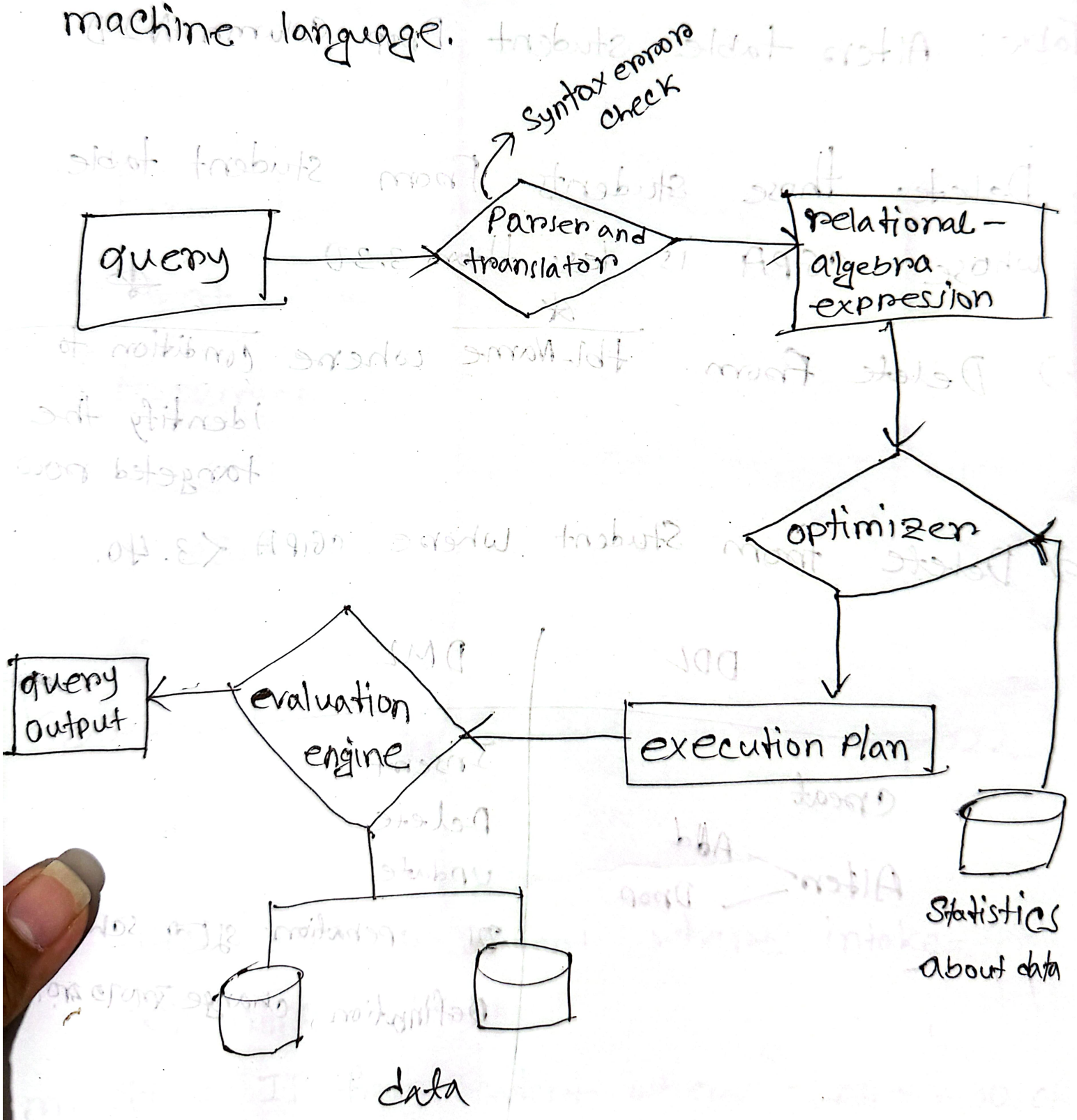
⇒ Delete from tbl-Name where condition to identify the targeted row

⇒ Delete from Student where CGPA < 3.40.

DDL	DML
Creat	Insert
Alter — Add	Delete
Alter — Drop	Update
	ER operation ছাড়া schema Defination change করতে পারবে

Query Processor

Query Processing is the process of translating/converting high level SQL into low level machine language.



Database Users

4 types of user:

- Native Users: Unsophisticated users who interact with the system by invoking one of the application programs that have been written previously.
- Application programmers: are computer professionals who write application programs.
- Sophisticated users: interact with the system without writing programs.
- Specialized users: write specialized database applications that do not fit into the traditional data-processing framework. For example CAD, graphic data, audio, video.

Database Admin

1. Schema definition
2. Storage structure and access-Method definition
3. Schema and physical-organization modification
4. Granting of authorization for data access
5. Routine ~~maintain~~ maintenance.

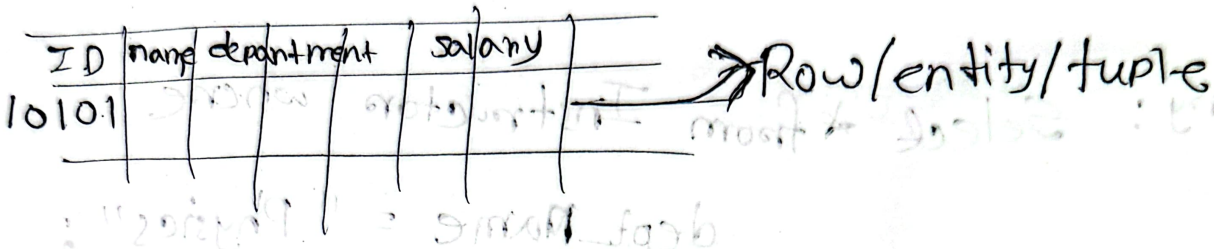
4. Periodically backing up the database.

5. Ensuring that enough free disk space is available for normal operations and upgrading disk space as required.

6. Monitoring jobs running on the database and ensuring that performance is not degraded by very expensive tasks submitted by some users.

Chapter 2

Intro to Relational Model



Relational Algebra

Six basic operators

1. Select: σ
2. Project: Π
3. Union: $\cup$
4. Intersection: $\cap$
5. Set difference: $-$
6. Cartesian Product: $\times$
7. rename: ρ

Select operation: Selected Row and all columns.

a. Select those tuples of the instructor relation where the instructor is in the "physics" department.

⇒ Query: Select * from Instructor where

dept_Name = "Physics";

RA ⇒ $\sigma_{condition}(table\text{-}Name)$

$\sigma_{dept_name="Physics"}(Instructor)$

b. Fnd the Instructors in Physics with a salary greater $90000 we write:

Query: Select *from Instructor where dept-Name = "Physics"

AND salary > 90000;

RA =

$\sigma_{dept_Name="Physics" \wedge Salary > 90000}(Instructor)$

Project :- Π

Selected columns and all rows.

Q. Show the Id, name and salary of each instructor

query: Select Id, Name, Salary from Instructor.

$$RA = \Pi_{Id, Name, salary}(Instructor)$$

Q Find the names of ~~those~~ all instructors in the Physics department.

⇒ Select Name from Instructor where dept_Name = "Physics";

$$RA = \Pi_{Name}(\sigma_{deptName = "physics"}(Instructor))$$

Q. Find the name and Id of those students whose CGPA is greater than 3.50 and age is greater than 22.

Q⇒ Select name, ID From student where CGPA > 3.50 AND age > 22;

$$RA = \Pi_{name, ID}(\sigma_{CGPA > 3.50 \wedge age > 22}(student))$$

LAB2

Q.9

Q.11

"_0%"

% = ignore

Q.12. "%0%

Q.13. Select customers_name, Length (Customer_name) "Length";

Q.14

Substr (customers_name 1, 3)

Adams

(Select customers_name, Substr (customers_name, 1, 3) from customers;)

f. $\Pi_{courseTitle}(course)$

h. $\Pi_{Name}(\sigma_{CGPA>3.50}(Student))$

k. $\Pi_{Name}(\sigma_{Age>22 \wedge CGPA>3.50}(Student))$

l. $\Pi_{St_ID}(\sigma_{intake=44 \vee CGPA=3.06}(Student))$

F. $\Pi_{courseTitle}(course)$

h. $\Pi_{Name}(\sigma_{CGPA>3.50}(student))$

k. $\Pi_{Name}(\sigma_{Age>22 \wedge CGPA>3.50}(student))$

l. $\Pi_{St_ID}(\sigma_{intake=44 \vee CGPA=3.06}(student))$

F. $\Pi_{courseTitle}(course)$

h. $\Pi_{Name}(\sigma_{CGPA>3.50}(Student))$

k. $\Pi_{Name}(\sigma_{Age>22 \wedge CGPA>3.50}(Student))$

l. $\Pi_{st_ID}(\sigma_{intake=44 \vee CGPA=3.06}(Student))$

Union = U

$A = \{1, 2, 3\}$. $B = \{4, 2, 5\}$

$A \cup B = \{1, 2, 3, 4, 5\}$

Actor

Name	Gender
AJ	M
JK	M
HA	M
AB	F
PN	F

U

Singer

Name	Gender
TK	M
HA	M
TF	F

⇒

Name	Gender
AJ	M
JK	M
HA	M
AB	F
PN	F
TK	M
TF	F

⇒SQL:

(Select Name, Gender from Actor) ~~U Select~~ Union (Select Name, Gender from Singer)

R.A:

$\Pi_{Name, Gender}(Actor) \cup \Pi_{Name, Gender}(Singer)$.

Conditions for performing Union operation:

1. 2 tables must have same attributes
2. The attributes domains must be compatible

Intersection

Actor

Name	Gender
AJ	M
JK	M
HA	M
AB	F
PN	F

$\cap$

Singer

Name	Gender
TK	M
HA	M
TF	F

Conditions for performing Union operation same as Union.

Name	Gender
HA	M

SQL: (Select Name, Gender from Actor) Intersect (select Name, Gender from Singer)

R.A: $\Pi_{Name, Gender}(Actor) \cap \Pi_{Name, Gender}(Singer)$.

Set difference

Name	Gender
AJ	M
Jk	M
HA	M
AB	F
PN	F

—

Name	Gender
Jk	M
HA	M
TF	F

⇒

Name	Gender
AJ	M
Jk	M
AB	F
PN	F

Sql : (Select Name, Gender From Actor

Minus

(Select Name, Gender from Singer)

RA : $\Pi_{Name, Gender}(Actor)$ —

$\Pi_{Name, Gender}(Singer)$

Cartesian Product

$A = \{a, b\}$, $B = \{1, 2\}$

$(A \times B) = \{(a,1), (a,2), (b,1), (b,2)\}$

Student

Id	Name
1	A
2	B
3	C

X

Home

Id	House No.
1	A-11
2	B-12
5	E-15

⇒

Student. ID	Name	Home. ID	House No.
1	A	1	A-11
1	A	2	B-12
1	A	5	E-15
2	B	1	A-11
2	B	2	B-12
2	B	5	E-15
3	C	1	A-11
3	C	2	B-12
3	C	5	E-15

Q. Find the name of student who lives in B-12. where House-NO = 'B-12', AnD Student.ID Home id.

~~26 28605453 40~~

Select Student.Id, Name, Home.Id, House_No,
From Student, Home where
House_No = "B-12" AND Student ID = Home.ID;

RA:

$\Pi_{Name}(\sigma_{House_NO="B-12" \wedge Student.Id = Home.Id}(Student \times Home))$

Natural Join ($\bowtie$)

Student

ID	Name
1	A
2	B
3	C

$\bowtie$

Home

ID	House
1	A-11
2	B-12
5	E-15

$\Pi_{Name, House_NO. = "B-12"}$

$\Rightarrow$

ID	Name	HouseNo
1	A	A-11
2	B	B-12

SQL: Select Name from Student NATURAL JOIN Home where House_NO = "B-12";

~~House_NO = "B-12~~

Relational Algebra (RA):

$$\Pi_{Name}\left(\sigma_{HouseNO = "B-12"}(Student \bowtie Home)\right)$$

Q.

Loan

L_no	b_name	amount
L-11	RH	900
L-14	DT	1500
L-15	PR	1500
L-16	PR	1300
L-17	DT	1000

Borrower

C_name	L_no
Adams	L-16
Smith	L-11
Jones	L-93
Jackson	L-14

Q1. Perform Natural Join (O/P + SQL + RA)

Q2. Find the name of those customers whose loan ~~amou~~ amount is greater than 1000 (SQL + RA)

Q.1. : Natural Join ⋈

loan

L-NO	b_name	amount
L-11	RH	900
L-14	DT	1500
L-15	PR	1500
L-16	PR	1300
L-17	DT	1000

⋈

c_Name	L-NO
Adams	L-16
Smith	L-11
Jones	L-93
Jackson	L-14

⇒

L_no	b_name	amount	c_Name
L-11	RH	900	Smith
L-14	DT	1500	Jackson
L-16	PR	1300	Adams.

L_no, b_name, amount, c_name | * from

SQL: Select * from Loan Natural Join Borrower;

RA : Loan ⋈ Borrower

Q.2

SQL : Select C_name From Borrower

Natural Join Loan where ~~Amont~~ Amount > 1000;

RA :

$$\Pi_{C\text{-}name}\left(\sigma_{Amount>1000}(Loan \bowtie Borrower)\right)$$

Left Outer Join $(\ldots \bowtie \ldots)$

⇒

Loan

L-NO	b_name	amount
L-11	RH	9000
L-14	DT	1500
L-15	PR	1500
L-16	PR	1300
L-17	DT	1000

$\bowtie$

Borrower

C-Name	L-NO
Adams	L-16
Smith	L-11
Jones	L-03
Jackson	L-14

L-no	b-name	amount	C-name
L-11	RH	900	Smith
L-14	DT	1500	Jackson
L-15	PR	1500	Null
L-16	PR	1300	Adams
L-17	DT	1000	Null

SQL: Select * from loan NATURAL LEFT OUTER JOIN Borrower;

RA: $\Pi_{\text{Loan_number}, \text{b-name}, \text{amount}, \text{C-name}}$ (Loan ⟕ Borrower)

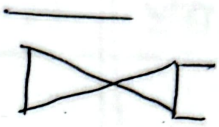
Right outer Join (⟖)

loan

L-no	b-name	Amount
L-11	RH	900
L-14	DT	1500
L-15	PR	1500
L-16	PR	1300
L-17	DT	1000

Borrower

c-name	L-no
Adams	L-16
Smith	L-11
Jones	L-93
Jackson	L-14



Loan_no	c-name	b_name	Amount
L-16	Adams	PR	1300
L-11	Smith	RH	900
L-093	Jones	Null	Null
L-14	Jackson	DT	1500

SQL: Select * from loan NATURAL RIGHT OUTER JOIN Borrower;

Full outer Join (⟗)

L-no	C-name	bname	Amount
L-11	Smith	RH	900
L-14	Jackson	DT	1500
L-15	Null	PR	1500
L-16	Adams	PR	1300
L-17	Null	DT	1000
L-93	Jones	Null	Null

keys

Student (St_Id, Name, Dept, CGPA)

ID	Name	Dept	CGPA

keys is an attribute/set of attributes for uniquely identifying each tuple/row.

4 types of keys:

→ 1. Superkey :- যে attribut এ column এ information
{Id} {Email} {ID, Name} {Id Dept}
{Email, name} {Email, dept} {Id, Name, Email, dept}

→ 2. Candidate key

→ 3 Primary key

→ 4. Foreign key

Candidate key:

- A candidate key is a minimal super key : it uniquely identifies a row and has no unnecessary attributes
- A table can have multiple candidate keys
  - {Id}
  - {Email}
  - .

Minimal super key. মানে হচ্ছে যে attribute carry করবে

Every candidate key can be a super key but each super key is not a candidate key.

primary key:

- The primary key is the candidate key selected to be the main identifier of rows.
- It must be unique and cannot contain NULL values.

=we can take one key from candidate key

Foreign key:

- A foreign key is an attribute (or set of attributes) in one table that refernance and primary key or uniaue key in another table.
- It helps establish relationships between tables and supports referential integrity.

Student (Id, Email, Name, Dept_Id)

Department (Dept_Id, Dept_name, location)

Student (Id, Name, Email, dept_Id)
PK, FK

Department (Dept_id, Dept_Name, location)
PK

SQL for defining primary key & Foreign key:

⇒ Create table student (Id int(10) PRIMARY KEY, Name varchar(30), Email varchar(25), dept_id varchar(10), FOREIGN KEY (dept_Id) REFERENCES Department(dept_id));

⇒ Create table Department (Dept_ID varchar(10) PRIMARY KEY, Dept_name varchar(10), location varchar(15));

Defining PK after creating a table:

Alter table Department ADD PRIMARY KEY (Dept_id);